

Python AI 서버 파이프라인 단계별 시각화

C++ 전처리 서버의 출력(512×512×3)이 Python AI 서버를 통과하면서 어떻게 변환되어 EfficientNet-B2에 입력되는지 확인합니다.

이미지 생성 방법은 [README.md](#) 참조

C++ 전처리 단계는 [preprocess-server.md](#) 참조

파이프라인 개요

512×512×3 이미지 (C++ 출력)

- |
- ▼ [1] PIL RGB 변환 + 채널 분리 시각화
- ▼ [2] Resize → 260×260
- ▼ [3] ToTensor + ImageNet Normalize
 - | → (1, 3, 260, 260) float32 numpy
- ▼ [4] EfficientNet-B2 Backbone
 - | → 1408-dim feature vector
- ▼ [5] 4개 Linear Heads
 - | → (head_a:19, head_b:14, head_c:16, head_d:11) logits
- ▼ [6] Sigmoid + Threshold(0.5)
 - | → 60개 문항 이진 결과
- ▼ [7] IQ 산출 (규준표)
 - | → IQ (67~133), 백분위 (1~99)

코드 위치: [ai-server/src/routes/analyze.py](#), [ai-server/src/core/preprocessing.py](#)

Stage 01: AI 서버 입력 이미지 (512×512)

이미지



- **형식:** RGB, 512×512×3, uint8
- **좌측:** 3채널 병합 원본 / **우측 3개:** 채널별 분리

채널 의미:

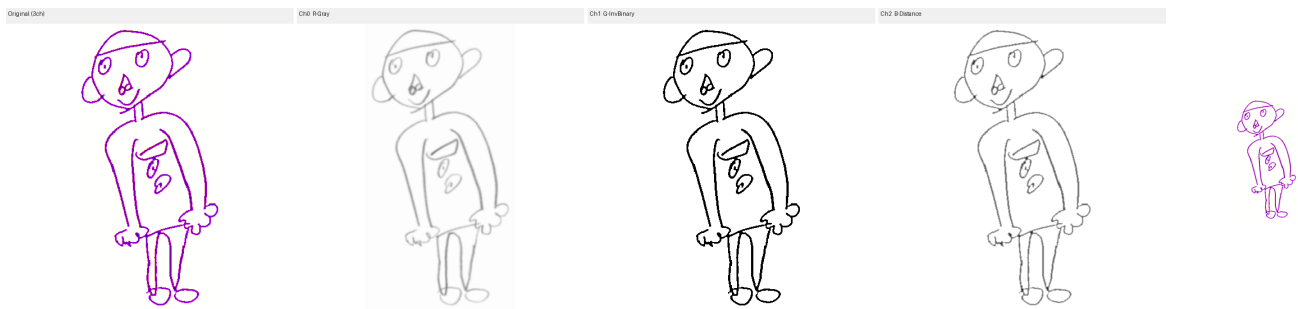
- Ch-0 R: 그레이스케일 (명도 / 필압 정보)
- Ch-1 G: 반전 이진화 (윤곽선 / 형태 정보)
- Ch-2 B: 거리 변환 (선 굵기 / 간격 정보)

C++ `HybridPreprocessFilter::ConstructChannels()`가 생성한 3채널.
 자세한 배경은 [preprocess-server.md — Stage 04](#) 참조.

Stage 02: Resize (260×260)

입력

출력



- 동작: `PIL.Image.resize((260, 260), BILINEAR)`
- 출력: 260×260×3, uint8

Why 260인가?: EfficientNet-B2의 기본(권장) 입력 크기가 260×260.

B0~B7 모델은 각각 다른 최적 해상도를 가지며, B2는 260에서 정확도-연산량 균형이 가장 좋다.

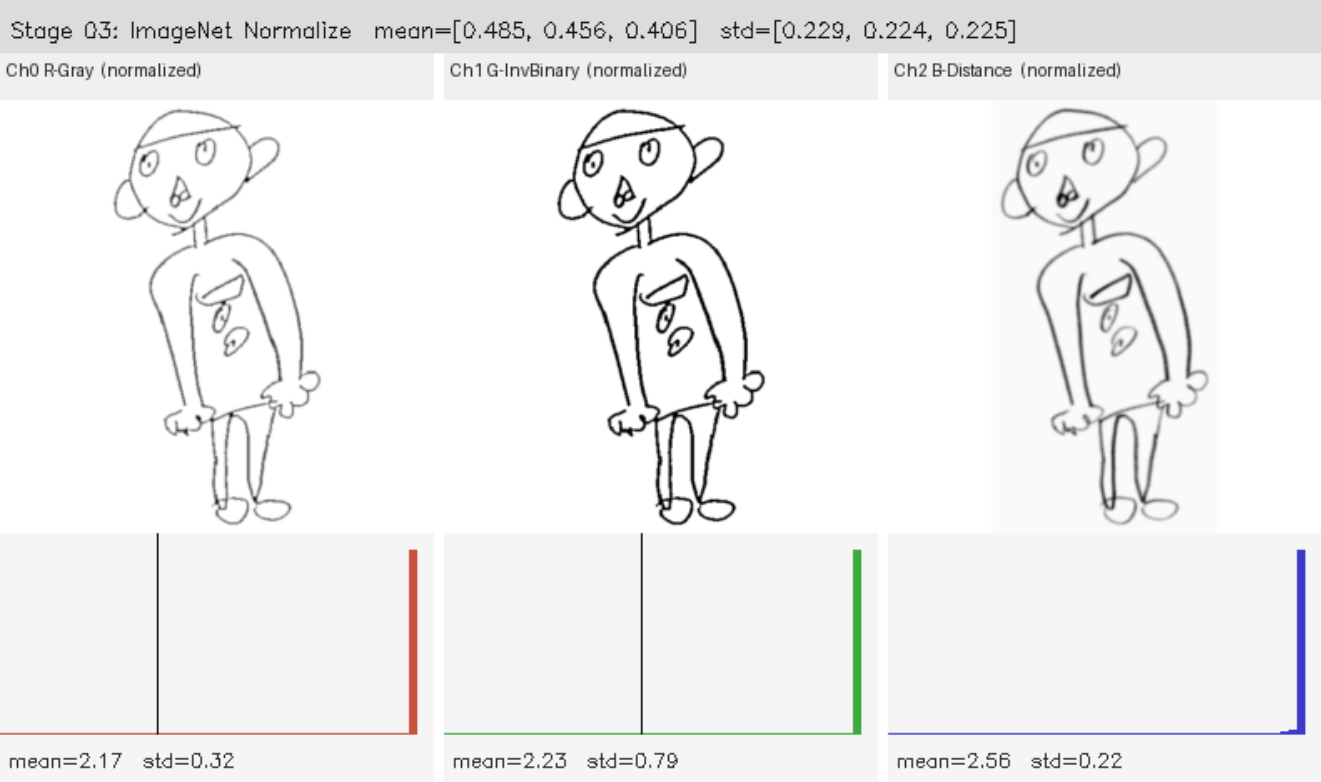
C++ 전처리에서 512로 만든 이유는 품질 손실 없이 다운샘플링하기 위함이고,
 최종 모델 입력 직전에 260으로 줄인다.

코드: [ai-server/src/core/preprocessing.py](#)

Stage 03: ImageNet 정규화

이미지

이미지



- 동작: $(\text{pixel} / 255.0 - \text{mean}) / \text{std}$
 - mean = [0.485, 0.456, 0.406]
 - std = [0.229, 0.224, 0.225]
- 출력: (1, 3, 260, 260), float32, 값 범위 약 [-2.5, 2.5]

상단: 각 채널을 min-max 스케일링으로 가시화 (원래는 음수 포함)

하단: 채널별 픽셀 값 분포 히스토그램

Why ImageNet 통계인가?: EfficientNet-B2는 ImageNet(RGB 자연 이미지)으로 사전학습되었다. 전이학습 시 입력 분포를 사전학습 때와 동일하게 맞춰야 backbone의 가중치가 올바르게 작동한다. 비록 우리 입력이 자연 이미지가 아닌 연필화이지만, 채널 구조(3채널 0~255)는 동일하므로 동일한 정규화 파라미터를 적용하는 것이 표준 관행이다.

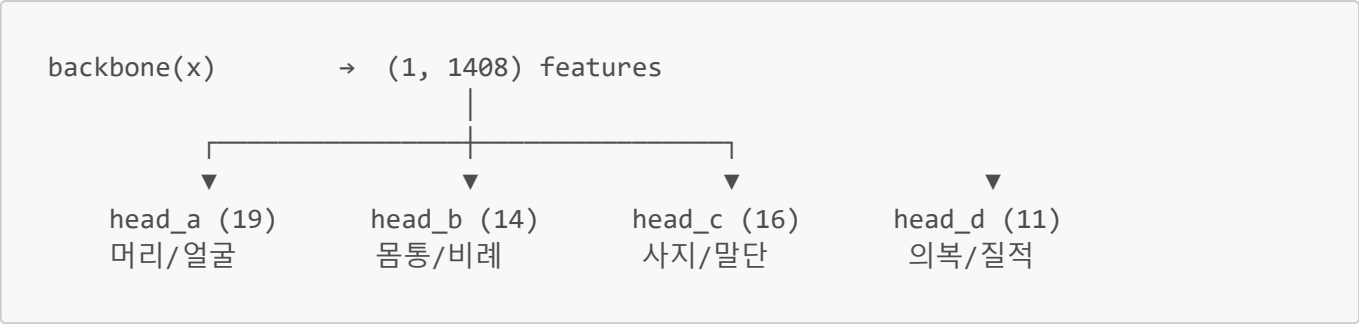
코드: [ai-server/src/core/preprocessing.py](#)

Stage 04~07: 모델 내부 (시각화 불가)

이후 단계는 이미지가 아닌 고차원 텐서/벡터로 변환되므로 이미지로 시각화하지 않는다.

Stage	입력	출력	설명
04	(1, 3, 260, 260) float32	(1, 1408) float32	EfficientNet-B2 backbone feature
05	(1, 1408)	4개 logit 벡터	4개 Linear Head 독립 분류
06	logits	60개 이진값	Sigmoid → threshold 0.5
07	60개 이진값 + 아동 정보	IQ, 백분위	규준표 조회

EfficientNet-B2 Head 구조



- 4개 head는 독립적으로 동일 feature에서 분기
- 각 head 출력은 `logit` → `sigmoid` → 0/1 (문항 점수)
- 전체 60문항 합계 = `raw_score` → IQ 산출

코드: [ai-server/src/core/model.py](#) — `HFDCClassifier`

정리 — 단계별 입출력 요약

Stage	동작	입력 형태	출력 형태	핵심 이유
01	(C++ 출력)	uint8 512×512×3	uint8 512×512×3	3채널 정보 통합
02	PIL Resize	uint8 512×512×3	uint8 260×260×3	EfficientNet-B2 권장 입력 크기
03	ImageNet Normalize	uint8 → float32	float32 (1,3,260,260)	사전학습 분포 일치
04	EfficientNet-B2	(1,3,260,260)	(1,1408)	이미지 → feature vector
05	4x Linear Heads	(1,1408)	4개 logit 벡터	문항별 이진 분류
06	Sigmoid+Threshold	logits	60개 이진값	확률 → 점수
07	IQ 산출	60개 점수 + 아동정보	IQ (67~133)	규준 표준화